Image Indexing

Assignment 2

By Einar López Altamirano* and Victor Armando Canales Lima†

Master's Degree in Intelligent Systems (MUSI)
Course 11762. Image Indexing & Retrieval by Content
Professor Emilio García Fidalgo

University of the Balearic Islands (UIB)

May 17, 2026

1 Introduction

Building on the first assignment, where we addressed image retrieval based on local features, this second assignment focuses on indexing. The idea is to compare different methods that organize image databases, so that the nearest-neighbor search is more efficient rather than exhaustive. We implement and compare four different methods.

1. Randomized k-d Trees: partitions the descriptor space recursively by using multiple randomized trees.
2. Locality-Sensitive Hashing: uses hashing to project descriptors onto random hyperplanes so that similar vectors collide into the same bucket.
3. Bag-of-Words: quantizes local descriptors into a discrete visual vocabulary, representing each image as a histogram of visual words. It uses an inverted index, widely used in text-retrieval techniques.
4. HNSW: builds a navigable graph over the data and is complemented with deep CNN features, representing a modern learning-based approach to image retrieval.

*einar.lopez1@estudiant.uib.cat
†victor-armando.canales1@estudiant.uib.cat

The four of them are evaluated on the INRIA Holidays dataset using mean Average Precision and build and query times. This report aims to analyze the accuracy, speed and memory trade-offs that distinguish tree-based, hash-based, vocabulary-based, and graph-based indexing.

2 Methods

In this section we present the four image retrieval pipelines implemented in this assignment. All four share the same evaluation protocol: an index is built from the database images, every query image is searched against the index to produce a ranked list of database images, and Mean Average Precision (mAP) is computed against the INRIA Holidays ground truth. The first three pipelines (Sections 2.2–2.4) operate on the precomputed SIFT descriptors of dimension 128 shipped with the dataset, capped at 400 descriptors per image for efficiency. The fourth (Section 2.5) operates on global ResNet-50 embeddings of dimension 2048.

2.1 Dataset and Evaluation Infrastructure

The evaluation was conducted using the INRIA Holidays dataset, specifically a "Mini" subset tailored for rapid development and testing. This dataset comprises distinct groups of personal holiday photographs designed to evaluate robustness to geometric and photometric transformations [1]. The first image of each group serves as the query, and the remaining images act as the relevant ground truth.

To facilitate the experimental pipeline, the provided `HolidaysDatasetHandler` class was utilised to load images, manage ground truth associations, and compute the mean Average Precision (mAP). A perfect retrieval system scores 1.0, while random retrieval scores close to 0.

2.2 Randomized k-d Trees

The first pipeline indexes the database SIFT descriptors with a forest of *randomized* k-d trees [2]. Each tree partitions the 128-D descriptor space recursively along a single coordinate axis; the splitting axis is chosen at random from the small set of dimensions with highest variance, so that different trees produce different partitions and together approximate exhaustive nearest-neighbour search. Queries are answered by descending all trees in parallel, intersecting the resulting candidate sets, and selecting the best match using a bounded backtracking budget.

We use the FLANN implementation [3] wrapped by `cv2.FlannBasedMatcher`, with `FLANN_INDEX_KDTREE` as the algorithm selected (algorithm = 1). The matcher accepts the per-image descriptor arrays directly and records the image index of every descriptor internally; an auxiliary `image_map` list translates back from FLANN's `imgIdx` field to the original filename.

Listing 1: Construction of the randomized k-d tree index using FLANN.

```
index_params  = dict(algorithm=1, trees=num_trees)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
flann.add(descriptors_list)   # list of (N_i, 128) arrays, one per image
flann.train()
```

At query time we retrieve the k nearest neighbours of every query descriptor and apply Lowe's ratio test [4] to filter ambiguous matches: a match is kept only when the distance to the best neighbour is significantly smaller (a factor 0.75) than the distance to the second best. Each surviving match contributes one vote to the image from which its neighbour originated, and the database images are ranked by vote count.

Listing 2: Lowe's ratio test and vote-based ranking.

```
matches = flann.knnMatch(query_descs,k=k)
for match in matches:
    if len(match) >= 2:
        m, n = match[0], match[1]
        if(m.distance < ratio_test*n.distance):
            matches_counts[m.imgIdx] += 1
ranked_image_ids = [img_idx for img_idx, count in matches_counts.most_common()]
```

2.3 Locality-Sensitive Hashing (LSH)

The second pipeline replaces the spatial-partitioning index with a hash-based one, using LSH [5] as implemented by FAISS's [6]. Each SIFT descriptor $\mathbf{x} \in \mathbb{R}^{128}$ is projected onto a fixed set of `nbits` random hyperplanes drawn from a Gaussian distribution, an the sign of each projection becomes one bit of a compact binary code.

Descriptors that are close in Euclidean space tend to fall on the same side of any random hyperplane, so Hamming distance between codes approximates angular similarity in the original space. Search reduces to fast bitwise operations and is effectively independent of the original dimensionality.

Listing 3: LSH index construction with FAISS.

```
lsh_index = faiss.IndexLSH(d, num_bits)
for image in database_images:
    curr_descriptors = dataset.get_descriptors(image)
    if curr_descriptors is None or len(curr_descriptors) == 0:
        continue
    curr_descriptors = curr_descriptors[:max_descriptors]
```

```
    lsh_index.add(curr_descriptors)
    image_map.extend([image] * len(curr_descriptors))
```

Note that `image_map` is extended once per descriptor, so the FAISS row index of every bit-code can be mapped back to its source image. At query time we retrieve the k nearest binary neighbours of each query descriptor and accumulate per-image votes; the database images are returned sorted by total vote count.

Listing 4: Per-image voting from LSH search results.

```
distances, indices = lsh_index.search(descriptors, k)
votes = Counter()
for idx in indices.flatten():
    if idx == -1:
        continue
    image_name = image_map[idx]
    votes[image_name] += 1
ranked_dict[filename] = [img for img, _ in votes.most_common()]
```

2.4 Bag-of-Words (BoW)

The third pipeline implements the Bag-of-Visual-Words model [7]. Local descriptors are *quantised* into a finite vocabulary of *visual words*; each image is represented as a histogram of word occurrences; and similarity between images is measured as the cosine similarity of their (TF-IDF re-weighted) histograms.

We use the precomputed vocabularies of size $k \in \{200, 2000, 10000, 20000\}$ that ship with the dataset; they were trained offline by k-means on the independent Flickr60K corpus. The vocabulary is wrapped in a brute-force `faiss.IndexFlatL2`, so quantisation reduces to nearest-centroid search. Then a histogram of visual word occurrences is built alongside an inverted index for fast retrieval.

Listing 5: Descriptor quantisation, BoW histogram, and inverted index update.

```
_, indices = self.vocindex.search(descriptors, 1)
indices = indices.flatten()
self.image_histograms[image_name] = np.bincount(
    indices, minlength=len(self.vocabulary))
for word_id in np.unique(indices):
    self.inverted_index.setdefault(word_id, []).append(image_name)
```

Once every database image has been processed, raw counts are re-weighted with the standard (smoothed) TF-IDF scheme so that visual words appearing in many images contribute less to the similarity score. The +1 smoothing inside and outside the logarithm avoids division by zero for unseen words and a vanishing IDF for words that appear in every image.

Retrieval first quantises the query descriptors with the same FAISS index, optionally TF-IDF re-weights the resulting histogram, and uses the inverted index to assemble the set of candidate database images that share at least one visual word with the query. Each candidate is then scored by the cosine similarity between its (re-weighted) histogram and the query histogram, and the candidates are returned ranked by descending score.

Listing 6: TF-IDF re-weighting of the BoW histograms.

```
idf = np.zeros(vocab_size)
for word_id in range(vocab_size):
    num_images_with_word = len(self.inverted_index.get(word_id, []))
    idf[word_id] = np.log((total_images+1)/(num_images_with_word+1))+1
for key, value in self.image_histograms.items():
    tf = value/ value.sum()
    self.image_histograms[key] = tf * idf
```

2.5 Deep Learning Retrieval with HNSW

The fourth pipeline replaces the SIFT descriptors with global, semantically-meaningful image embeddings produced by a ResNet-50 [8] pretrained on ImageNet. Each image is mapped to a 2048-dimensional feature vector by the network's penultimate layer; the vectors are provided precomputed. Because the network was optimised for image classification, the geometry of its embedding space encodes semantic similarity, and standard distance metrics (cosine, Euclidean) become meaningful image-level similarity measures.

We L2-normalise both database and query vectors so that the inner product between any two of them equals their cosine similarity, and then index them with a Hierarchical Navigable Small World (HNSW) graph [9] as implemented by `faiss.IndexHNSWFlat`. HNSW is a multi-layer proximity graph in which each node is connected to at most M neighbours per layer; the top layer is sparse and serves as the entry point, while the bottom layer contains every database vector. Greedy traversal from the entry point locates the approximate nearest neighbours in $O(\log N)$ hops on average.

Listing 7: HNSW index construction with cosine similarity.

```
db_vectors = np.vstack(db_vectors).astype(np.float32)
faiss.normalize_L2(db_vectors)
d = db_vectors.shape[1] # Descriptor dimensionality (2048)
hnsw_index = faiss.IndexHNSWFlat(d, M, faiss.METRIC_INNER_PRODUCT)
hnsw_index.hnsw.efConstruction = efConstruction
hnsw_index.add(db_vectors)
```

Three parameters control the accuracy / speed trade-off of an HNSW index: M (the number of bidi-

rectional connections per node, fixed at construction time), `efConstruction` (the size of the candidate list used during insertion, which determines graph quality) and `efSearch` (the beam width at query time). Larger values of all three increase recall at the cost of build and search time. At query time the same L2 normalisation is applied to every query vector before it is searched against the index:

Listing 8: HNSW search with cosine similarity.

```
index.hnsw.efSearch = efSearch
for filename, descriptor in query_descs_dict.items():
    if descriptor is None:
        continue
    # Ensure the descriptor is a 2D float32 array: shape (1, 2048)
    query_vector = np.array(descriptor, dtype=np.float32)
    if query_vector.ndim == 1:
        query_vector = query_vector.reshape(1, -1)
    faiss.normalize_L2(query_vector)
    distances, indices = index.search(query_vector, k)
    ranked_list = []
    for idx in indices.flatten():
        if idx != -1:  # -1 indicates FAISS couldn't find enough neighbors
            ranked_list.append(image_map[idx])
    ranked_dict[filename] = ranked_list
```

Unlike the three SIFT-based pipelines, which retrieve at the level of individual local descriptors and aggregate votes per image, the deep-learning pipeline retrieves directly at the image level: each image is represented by a single 2048-D vector, and the k nearest neighbours of the query vector in this embedding space are themselves the k most similar database images. Consequently, neither Lowe's ratio test nor a per-image voting step is required.

3 Results and Analysis

4 Results and Analysis

This section presents the experimental outcomes obtained by following the protocol described in Section 2, organised task by task as in the accompanying notebook. The four indexing pipelines are evaluated on the same Mini-Holidays subset, and we comment on the figures and the comparative table embedded in this section as each task is discussed.

4.1 Randomized k-d Trees

Task 1 — Baseline FLANN k-d tree index. The first experiment evaluates the baseline configuration of the FLANN-based randomized k-d tree index, capping the number of descriptors per image at 400 and using 4 trees. Under this default setting the pipeline achieves a Mean Average Precision of **0.78509**, which establishes the reference point against which all subsequent SIFT-based experiments are compared.

Task 2 — mAP consistency across runs. To verify the determinism of the pipeline, five sequential experiments were run. When the trees are not rebuilt between runs, the mAP remains a constant 0.78509 across all five tests. When the index is rebuilt before each experiment, however, the mAP fluctuates between runs (e.g. 0.79298, 0.80000, 0.79825, etc.). The results are consistent when reusing the same trees because the search phase is purely deterministic, always traversing the exact same branches to find identical nearest neighbours. However, when the trees are rebuilt the mAP fluctuates because FLANN's k-d tree construction is stochastic: to build the tree, the algorithm randomly selects a splitting dimension from among those with the highest variance. The slight variations in performance occur because this randomness acts as a lottery, some random splits happen to partition the feature space more optimally for retrieval than others, slightly altering the accuracy of the neighbours found.

Task 3 — Effect of the number of trees. The third experiment sweeps the number of trees over $\{1, 2, 4, 8, 16, 32\}$. The resulting table shows that mAP stays relatively stable, between roughly 0.78 and 0.82, while both build time and average search time per query grow steadily with the tree count. Figure 1 visualises this behaviour with three side-by-side bar plots of mAP, average search time, and build time against the number of trees. The visualisation confirms that adding more trees drastically scales up the build and search times but provides little to no improvement in mAP. Increasing the number of trees shows minimal increase for mAP, while both build and search times continue to scale linearly. This could indicate that a small forest is entirely sufficient to capture the spatial variance of the descriptors in this dataset, meaning that adding more trees simply introduces redundant partitions rather than new, valuable information. Furthermore, because approximate search algorithms like FLANN typically limit the total number of nodes checked during a query, having too many trees might force the algorithm to spread its search budget too thinly across multiple roots, increasing the traversal overhead without actually improving the quality of the nearest neighbours found.

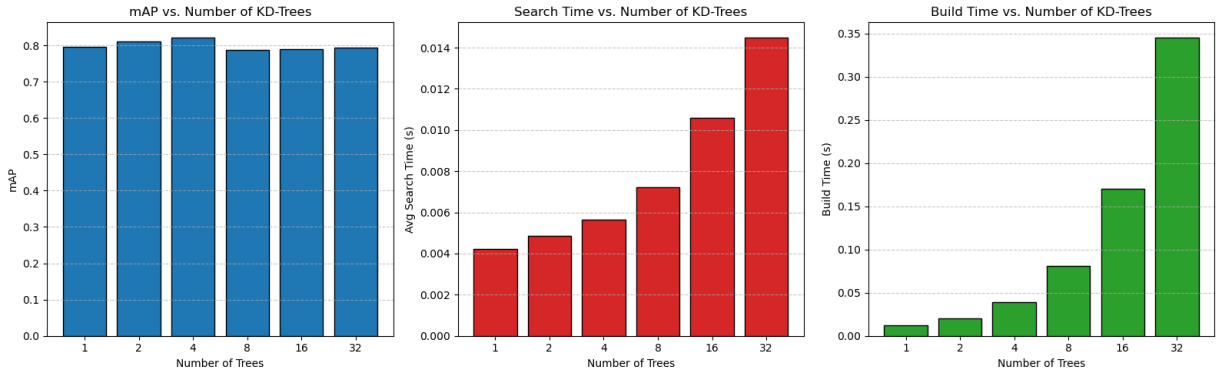

Figure 1: Comparison of k-d tree approach performance by number of trees

Task 4 — Effect of the number of descriptors per image. The fourth experiment instead varies the maximum number of descriptors per image over $\{50, 100, 200, 400, 800, 1600\}$. The table shows that mAP, build time, and search time all rise significantly as more descriptors are kept. Figure 2 confirms this with three bar plots displaying mAP, average search time, and build time against the maximum number of descriptors per image. In contrast to the tree-count sweep, these plots reveal a clear positive correlation across all three metrics. Increasing the number of descriptors per image drastically improves mAP but comes with a steep computational cost, as both build and search times increase almost exponentially. It is possible that the initial surge in accuracy (up to 400 descriptors) occurs because the algorithm captures the most salient, highly distinctive visual features of the image. However, as we push to 800 and beyond, the k-d tree index size balloons and the algorithm is forced to process weaker, less reliable background features. While brute-forcing 1600 descriptors eventually achieves an impressive mAP, the severe penalty in search time suggests that this approach scales poorly, making a smaller set of high-quality keypoints a much more efficient strategy.

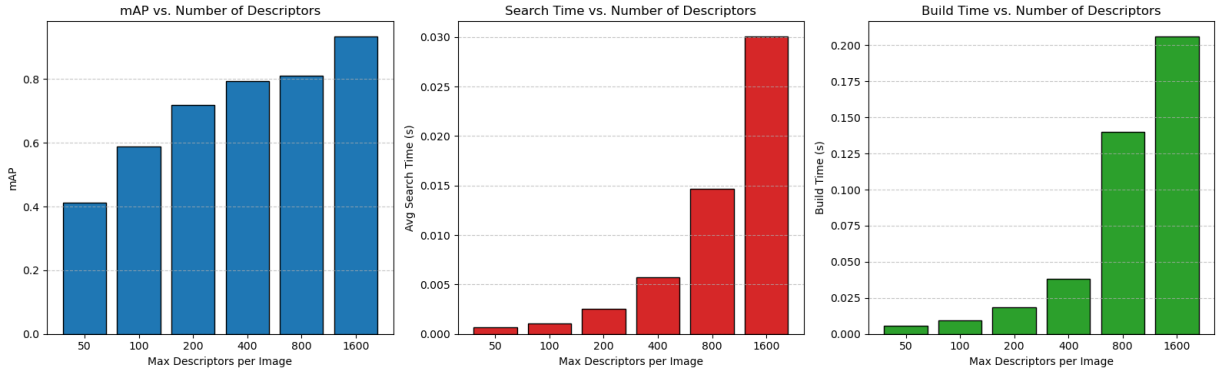

Figure 2: Comparison of k-d tree approach performance by number of descriptors

4.2 Locality-Sensitive Hashing

Task 5 — Baseline FAISS LSH index. The baseline LSH configuration uses the FAISS `IndexLSH` with 128 random hyperplanes (i.e. 128-bit codes). Under this initial configuration the pipeline reaches a mAP of **0.61619**, noticeably below the k-d tree baseline of Task 1.

Task 6 — Effect of the hash size `nbits`. A sweep over `nbits` from 4 to 512 reveals that the number of bits has a strong effect on retrieval accuracy: as `nbits` grows, the mAP rises consistently from 0.05592 at 4 bits to 0.81915 at 512 bits, with build and average query times remaining very low and with only minor fluctuations. Figure 3 displays these three quantities side by side: a line graph showing the steep and continuous upward trend of mAP, plus two bar plots showing that build time and average query time fluctuate without a discernible pattern as the hash size changes. The trend has not plateaued at 512 bits, which suggests that even more bits could still improve accuracy.

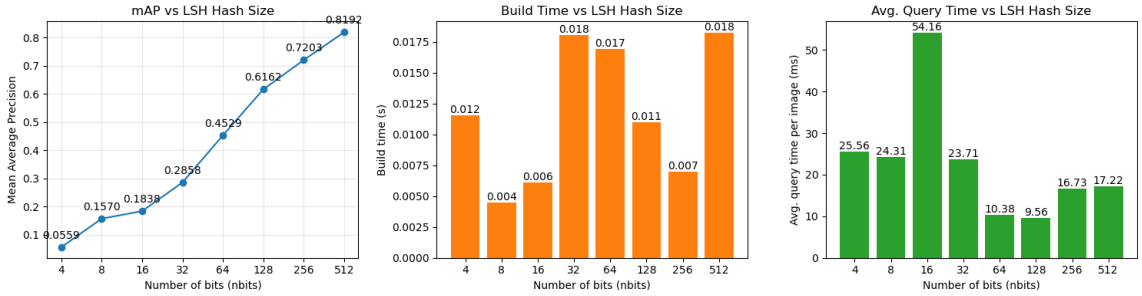

Figure 3: Comparison of LSH approach performance by number of bits

This behaviour is expected from LSH's logic. Each descriptor is hashed into a binary code of `nbits` bits, one bit per random hyperplane, so the number of available buckets is $2^{\texttt{nbits}}$. With only 4 bits there are just 16 buckets, and the entire 128-dimensional descriptor space is collapsed into them, causing very different descriptors to collide into the same code. As bits are added, the space is refined and the collision rate roughly halves with each additional bit, which is why accuracy grows consistently with `nbits`. The build time remains very small throughout, between 3 and 50 ms, increasing only slightly with `nbits`: building an LSH index only requires generating the random hyperplanes and hashing the database descriptors, so there is no data-dependent training step. The average query time, in turn, varies between 20 and 50 ms without a clear pattern; in theory longer codes require slightly more work per Hamming-distance comparison, but the effect is too small to see here, and the query time can best be described as approximately constant.

In summary, increasing `nbits` improves accuracy at a negligible cost in build and query time. The real trade-off is memory: each descriptor is stored as a code of `nbits` bits, so memory grows linearly with the hash size, and a 512-bit configuration consumes 128 times the memory of a 4-bit one. On this dataset there is no accuracy optimum, since larger codes are always better, and the only practical limit on `nbits` is memory consumption.

Task 7 — k-d trees vs. LSH. Figure 4 directly compares k-d trees and LSH. Three bar charts first contrast the two methods on their default settings, showing that k-d trees achieve a higher mAP and a faster query time, while LSH builds its index much faster. Two line plots then track the hyperparameter sweep trade-off curves for both methods (accuracy vs. query time and accuracy vs. build time), making it clear that k-d trees are much more stable in query time than LSH. Overall, k-d trees show better performance than LSH: even though the build time is considerably larger for the trees, both the search time and the mAP they reach are better. When varying the parameters (number of trees or `nbits`), k-d trees are much more stable in query time, and the mAP they obtain is not very different regardless of the chosen number of trees. In contrast, the LSH query time is much more unstable, and the achieved mAP is also strongly dependent on the parameter selection.

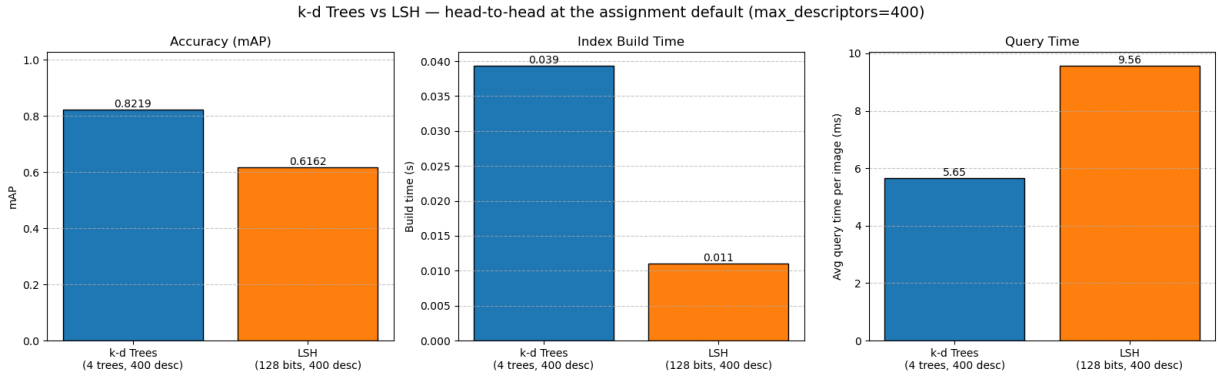

Figure 4: Comparison between LSH and k-d trees

4.3 Bag-of-Words

Task 8 — BoW across vocabulary sizes. The Bag-of-Words pipeline is evaluated over four different visual vocabulary sizes (200, 2k, 10k, and 20k). The mAP ranges from roughly 0.69 to 0.73, while both the training time and the query time per image increase drastically as the vocabulary grows.

Task 9 — Effect of vocabulary size on mAP. Figure 5 plots mAP on a truncated y-axis against the four vocabulary sizes. The curve is relatively flat, with a slight performance dip at 2k and 10k and a recovery at 20k. A larger vocabulary is therefore not always advantageous: all four sizes achieve a similar mAP, with only a $\sim$0.04 difference between the best (200) and worst (10k) performing sizes.

This can be understood through a trade-off. With a small vocabulary, visual words are coarse: many different descriptors collapse into the same word, so the words are less discriminative (different objects may share a word). Matching, however, is robust, since two photos of the same scene reliably quantise to the same words. With a large vocabulary the opposite happens: words are fine-grained and discriminative, but quantisation becomes fragile, as a descriptor from the same physical point seen under slightly different lighting may fall into different neighbouring words. Matching images then share fewer words, lowering

similarity. This is quantisation error, and it grows as the vocabulary gets finer. Ideally, this trade-off predicts a performance peak at some intermediate size. Our results do not show such a peak; if anything, the intermediate sizes perform worst. We attribute this not to a real effect but to the size of the dataset (∼49 images), where a 0.04 difference can be interpreted as noise. Vocabulary size must be matched to dataset size and diversity. On this mini dataset no size is clearly better, which confirms that simply increasing the vocabulary size does not guarantee better retrieval.

Task 10 — Effect of vocabulary size on time. Figure 6 compares training time and average query time across the four vocabulary sizes. The plots visually communicate an aggressive, linear-like cost in computing time as the system scales from 200 to 20k visual words. Vocabulary size therefore has a clear impact on the system: it increases compute cost but barely affects accuracy. As discussed in Task 9, mAP stays roughly flat across all four sizes (within noise), while response time grows steeply in both the training and the query phases. Training time rises from 0.27 s (200 words) to 6.62 s (20k words), and the average query time rises from 5.7 ms to 265.7 ms per image, a roughly 46× increase.

The reason for this growth is the index used for quantisation. The visual vocabulary is stored in a `faiss.IndexFlatL2`, which performs a brute-force nearest-neighbour search. Every descriptor is compared against every centroid, so the cost per descriptor is O(vocabulary size). Doubling the vocabulary doubles the number of comparisons, and since both training and querying quantise descriptors, both phases scale the same way. Increasing the vocabulary size buys no accuracy here but establishes a large, linearly-growing time cost: the 20k vocabulary is 46× slower per query than the 200 vocabulary for essentially the same mAP. If quantisation speed became a bottleneck, a non-exhaustive index over the centroids (e.g. HNSW) would avoid this linear growth, trading exactness for speed.

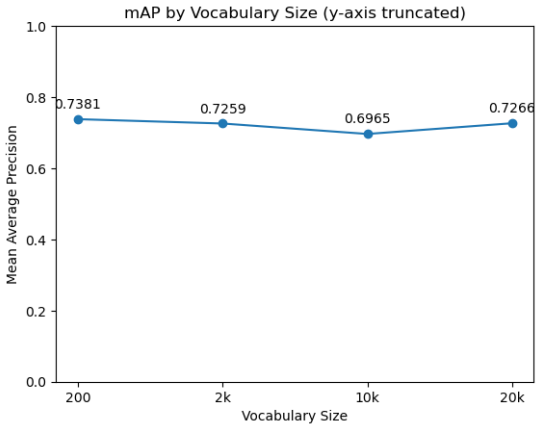

Figure 5: Precision performance of Bag-of-Words approach by vocabulary size

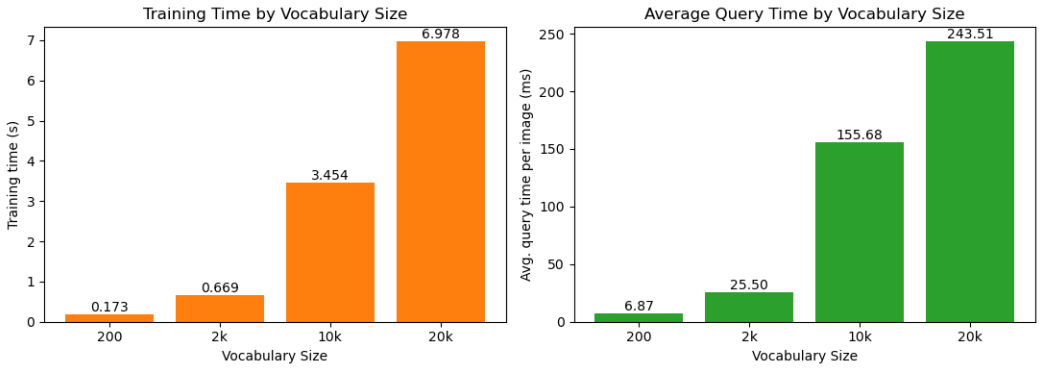

Figure 6: Time performance of Bag-of-Words approach by vocabulary size

Task 11 — Influence of the vocabulary training set. The results also depend on the set of images used to generate the vocabulary. As noted in the notebook, the vocabularies were trained with k-means on a separate dataset, Flickr60K, rather than on the Holidays dataset itself. The vocabulary is effectively a model of the visual content of its training images: if that training set has a different distribution of content than the target dataset, the centroids are positioned for the wrong distribution, and when Holidays descriptors are quantised they are assigned to centroids that do not represent them well, increasing quantisation error and degrading retrieval. The quality of the vocabulary therefore depends on how well its training set matches and covers the descriptor distribution of the target dataset.

Using the same dataset for vocabulary generation is not a solution either, since the test images would bias the evaluation. There is thus a trade-off between a fair evaluation and a close domain match. Retrieval performance could be improved in several ways: a domain-matched vocabulary trained on images representative of the target content would place centroids more accurately; soft (multiple) assignment, where each descriptor is assigned to its several nearest words rather than only one, would reduce the quantisation fragility discussed in Task 9; finally, stronger descriptors such as deep features would provide a more discriminative representation than raw SIFT.

Task 12 — TF-IDF vs. raw histograms. Table 1 summarises the mAP and time costs for models using TF-IDF weighting versus models using raw histograms across all four vocabulary sizes. The performance differences are negligible: TF-IDF has no real effect on retrieval performance for this dataset. Across the four vocabulary sizes, the change in mAP is at most 0.01: slightly higher with TF-IDF at 2k and 10k, slightly lower at 20k, and exactly the same at 200. Since a difference of $\sim$0.04 was already considered within noise in Task 9, these differences are also interpreted as noise rather than a real improvement or degradation.

TF-IDF is a corpus-statistics technique, and a 49-image corpus is far too small for those statistics to be reliable. On a larger dataset TF-IDF would likely show a clearer positive improvement, since the reweighting of the histograms helps to emphasise the words that are most discriminative between images, while reducing the impact of words that appear so often they carry little information. In terms of compute cost, TF-IDF

adds a small overhead to the training phase (the `apply_tfidf` step performs an extra IDF computation and a reweighting pass over all histograms), while query time is essentially unchanged, as reweighting the query histogram is just a fast element-wise multiplication. In summary, TF-IDF neither improves nor degrades results on this dataset, but adds a small (negligible) extra training cost.

Vocab	**TF-IDF**			**Raw**		
	mAP	**Train (s)**	**Query (ms)**	**mAP**	**Train (s)**	**Query (ms)**
200	0.73807	0.167	5.68	0.73807	0.138	8.63
2k	0.72588	0.838	28.18	0.72462	0.707	28.04
10k	0.69653	3.091	117.72	0.69298	3.098	117.24
20k	0.72661	6.388	233.36	0.73713	5.876	230.71

Table 1: Impact of TF-IDF on BoW Performance

4.4 Deep Learning Retrieval with HNSW

Task 13 — Baseline DL + HNSW. The Deep Learning pipeline, using ResNet-50 features indexed with a Hierarchical Navigable Small World (HNSW) graph, achieves a mAP of **0.98684** on the Mini-Holidays dataset, dramatically outperforming all SIFT-based pipelines.

Task 14 — HNSW profiles. A final experiment compares HNSW performance across three predefined profiles that span the accuracy/speed trade-off of the index. The *Fast* profile uses $M = 8$, `efConstruction` = 16, `efSearch` = 8 and $k = 10$, producing a sparse graph that is cheap to build and traverse; the *Balanced* profile uses $M = 32$, `efConstruction` = 64, `efSearch` = 64 and $k = 100$, increasing the connectivity and beam width for a richer search; and the *Accurate* profile uses $M = 128$, `efConstruction` = 256, `efSearch` = 256 and $k = 1000$, building a much denser graph and exploring a wider candidate list at query time. The accompanying table shows a consistent mAP of 0.98684 across the board, with extremely small times for both index building and average search. Figure 7 visualises mAP, average search time, and build time against the three HNSW profiles, highlighting that mAP remains at a static maximum and that, despite minor search-time increases in the *Accurate* profile, the overall computation cost is virtually unnoticeable.

The combination of deep feature embeddings and HNSW indexing drastically outperforms all classical solutions in both accuracy (0.987 mAP) and speed (∼0.02 ms per query). The mAP remains perfectly flat across all HNSW parameter profiles, most likely because the deep CNN embeddings represent the whole image context so effectively that the feature space is already highly clustered and separable. Consequently, even a very sparse, computationally cheap graph (the *Fast* profile) is entirely sufficient to navigate the space and find the exact nearest neighbours, making heavier graph constructions redundant for this dataset.

Furthermore, the speed of this strategy comes from evaluating just a single, dense semantic vector per image, completely eliminating the massive overhead of matching hundreds of local SIFT descriptors required by k-d Trees or LSH.

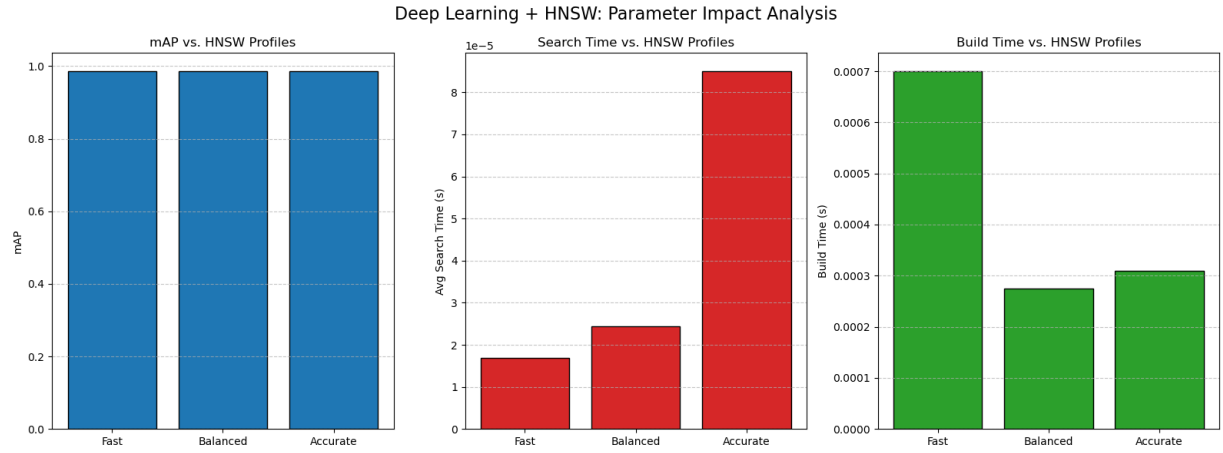

Figure 7: Performance of DL + HNSW approach under different parameter profiles

5 Conclusions

This assignment compared four different indexing paradigms for content-based retrieval. Among the methods based on SIFT descriptors, performance was similar and quite dependent on configuration: randomized k-d trees reached an mAP of 0.785 at the default setting and up to 0.934 with more descriptors, LSH ranged from 0.056 to 0.819 as the hash size grew, and Bag-of-Words remained around 0.73 regardless of vocabulary size. Each method exposed a different cost: k-d trees trade accuracy for build and query time, LSH for memory, and Bag-of-Words for an expensive quantization step. The clearest result, however, is that the HNSW index built on deep CNN features outperformed every other approach by a wide margin, achieving an mAP of 0.987 with near-instant search. This suggests that the choice of image representation, deep features against local descriptors has a greater impact on retrieval quality than the choice of indexing methods.

One important note is that the small size of the dataset (49 images) inflates the absolute scores and adds noise to the timing measurements, so the observed trends are more reliable than the exact values.

References

[1] H. Jegou, M. Douze, and C. Schmid, "Hamming embedding and weak geometric consistency for large scale image search," in *European conference on computer vision*, pp. 304–317, Springer, 2008.

[2] M. Muja and D. G. Lowe, "Scalable nearest neighbor algorithms for high dimensional data," *IEEE transactions on pattern analysis and machine intelligence*, vol. 36, no. 11, pp. 2227–2240, 2014.

[3] M. Muja and D. Lowe, "Flann-fast library for approximate nearest neighbors user manual," *Computer Science Department, University of British Columbia, Vancouver, BC, Canada*, vol. 5, no. 6, pp. 12–29, 2009.

[4] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International journal of computer vision*, vol. 60, no. 2, pp. 91–110, 2004.

[5] A. Gionis, P. Indyk, R. Motwani, *et al.*, "Similarity search in high dimensions via hashing," in *Vldb*, vol. 99, pp. 518–529, 1999.

[6] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *IEEE Transactions on Big Data*, 2025.

[7] Sivic and Zisserman, "Video google: A text retrieval approach to object matching in videos," in *Proceedings ninth IEEE international conference on computer vision*, pp. 1470–1477, IEEE, 2003.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[9] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.